

Informe Tarea 1

Kenji Kita Aspeé, kkita@usm.cl

Martín Corona Cardoza, mcorona@usm.cl

Benjamín Ríos Rodríguez, briosro@usm.cl

Felipe Rain Avila, frain@usm.cl

1. Parte 1

1.1. Diseño de la función lecturaSensor()

La función `lecturasensor()` está diseñada como una unidad de software independiente encargada de la adquisición de datos. En un sistema real, esta función interactuara directamente con el hardware.

Los requisitos operacionales de la función serían:

El rango que tiene para dar valores entrega datos entre 0 y 1000 mV, y el tipo de datos que entrega son números enteros.

El enum permite que el código sea autoexplicativo al usar etiquetas como INIT, READ y SHOW.

La figura 1 muestra que el programa inicia evaluando el valor que tiene enum, luego se realiza una escritura limpia (pone los valores a 0) para evitar datos residuales en la memoria.

En “lectura” se conecta con la función externa `lecturaSensor()`. Es el punto en el que el software “recibe” la simulación del mundo físico (0-1000 mV).

En “mostrar” se realiza una operación de salida, transformando los datos guardados en el arreglo en información legible para el usuario mediante la consola.

Independientemente de la tarea realizada, todos los caminos convergen en un único punto de salida. Esto garantiza que la función siempre termine de forma segura y devuelva el control al programa principal “main” para que este continúe con la siguiente fase del ciclo de vida de los sensores.

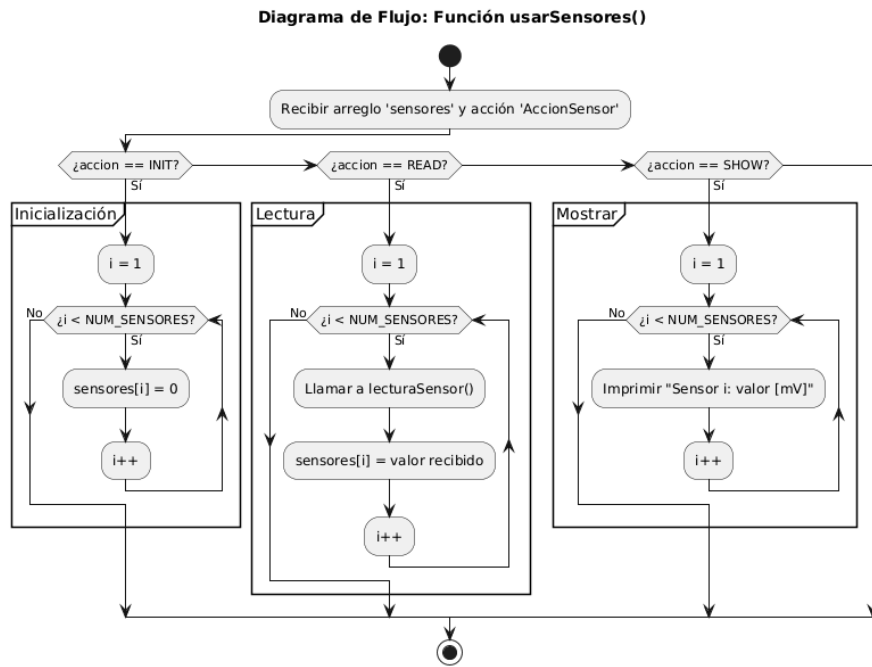


Figura 1

1.2. Diseño completo de la nueva función main()

Configuración de Entorno: Utiliza “srand((unsigned int) time(NULL))” para asegurar que las lecturas cambien en cada ejecución del programa.

Gestión de Memoria: Declara el arreglo de sensores con un tamaño definido por la constante “NUM_SENORES”.

Ciclo de Vida de la Aplicación: El “main” invoca a la función controladora “usarSensores()” en tres etapas consecutivas y bien diferenciadas:

- Inicialización (INIT): Limpia el arreglo asegurando que no contenga valores basura de la memoria.
- Captura de Datos(READ): Llena el arreglo invocando repetidamente a “lecturaSensor()”.
- Salida de Datos(SHOW): Procesa la información para mostrarla al usuario a través de la consola.

La figura 2 muestra el diagrama de flujo que representa la función principal de un programa para manejar datos de sensores en forma ordenada. Primero, prepara el sistema configurando una semilla para números aleatorios y creando una arreglo (lista) para guardar la información de seis sensores. Luego, el programa avanza por tres fases:

empieza por dejar todo en cero, sigue con una fase de captura donde se leen los datos reales, y termina con una salida que muestra esos resultados en pantalla. Finalmente, el programa avisa que todo salió bien y se cierra.

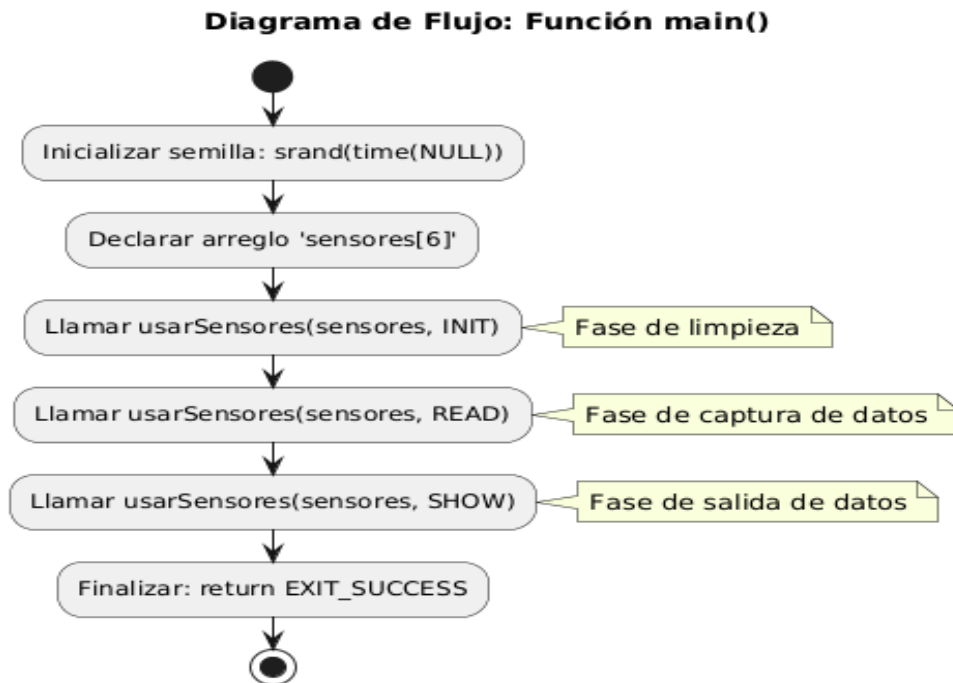


Figura 2

2. Parte 2

2.1. Hito 1

Responder todas las preguntas que se realizan en el enunciado

1) ¿Qué es lo que retorna la función `clock()`?

La función `clock()` no mide el tiempo como un reloj de pared (horas o minutos), sino que mide "clicks" o "ticks" de la CPU. Es un contador interno que va sumando cada vez que el procesador hace un pequeño esfuerzo desde que el programa empezó a correr. Básicamente, dice cuánta "atención" le ha dedicado el procesador al programa hasta ese momento.

2) ¿Qué unidades tienen las variables `inicio` y `fin`?

Sus unidades son pulsos de reloj (ticks). No son segundos ni milisegundos. En `inicio` marca cuántos pulsos llevaba el procesador cuando empezó el ciclo, y `fin` marca cuántos llevaba cuando terminó.

3) ¿Por qué se debe dividir la diferencia con `CLOCKS_PER_SEC`?

Porque cada computador es distinto; algunos procesadores son más rápidos y dan más "clics" por segundo que otros. La constante `CLOCKS_PER_SEC` nos dice cuántos de esos pulsos equivalen. Al dividir la diferencia (fin - inicio) por esa constante, transformamos esos "clics" abstractos en una unidad de tiempo real que los humanos podemos entender: los segundos.

4) ¿Qué hace (double) como prefijo de (fin - inicio)?

Este prefijo se llama "cast" o conversión forzada. Lo que hace es decirle al computador: *"Oye, aunque el resultado de esta resta sea un número entero, quiero que lo trates como un número con decimales (float)"*. Es cómo transformar un número cerrado (como 5) en uno con capacidad de tener decimales (cómo 5.0).

5) ¿Para qué sirve utilizar (double) como prefijo?

Sirve para no perder la precisión. En programación, si divides un número entero por otro entero, el computador "borra" los decimales y te da un resultado redondeado hacia abajo. Como el tiempo que estamos midiendo es extremadamente pequeño (nanosegundos), si no usamos (double), el resultado casi siempre terminaría siendo 0 porque el computador descarta todos los decimales importantes. Al usarlo, nos aseguramos de que la división sea exacta y nos muestre el tiempo más real posible.

2.2. Hito 2

La función `cicloPrueba()` funciona como un "cronómetro de precisión". Su objetivo es medir cuánto tiempo le toma al computador realizar tareas muy pequeñas, como imprimir un simple espacio o un asterisco en la pantalla.

Para que la medición sea justa, la función hace lo siguiente:

- Repetición: Como una sola impresión es demasiado rápida para que el reloj la note, la función repite la misma acción un millón de veces (1e8) usando un ciclo `for`.
- Comparación: Se mide el tiempo justo antes de empezar el ciclo y justo después de terminarlo. Al final, tomamos ese tiempo total y lo dividimos por el millón de repeticiones para saber cuánto tardó cada una por separado (el tiempo promedio).
- Decisión: Usamos instrucciones `if` para que el programa elija qué acción realizar (quedarse vacío, imprimir un espacio, un asterisco o un salto de línea) dependiendo de lo que queramos medir en ese momento.

Requisitos Operacionales:

La función debe cumplir con:

- Igualdad: Todos los casos deben repetirse exactamente la misma cantidad de veces.

- Limpieza: No debe haber otras tareas pesadas ejecutándose dentro de la función que puedan "ensuciar" el tiempo que estamos midiendo.
- Precisión: Al calcular el resultado, debe usar decimales (double) para que los nanosegundos no se pierdan por redondeos del computador.

Justificación del uso de enum caso_t:

En lugar de usar números secretos (como decir "el caso 0 es vacío, el caso 1 es espacio"), utilizamos un enumerado (enum) llamado caso_t. Esto se hace por tres razones principales:

1. Claridad: Es mucho más fácil leer `if (caso == ASTERISCO)` que leer `if (caso == 2)`. El código se entiende a simple vista sin tener que adivinar qué significa cada número.
2. Orden: Nos permite agrupar todas las opciones posibles bajo un solo nombre. Es como tener un menú cerrado donde sólo puedes elegir las opciones que nosotros definimos, lo que evita errores.
3. Eficiencia: Aunque para nosotros son palabras, para el procesador siguen siendo números muy simples, por lo que el computador no gasta energía extra procesándolos y la medición sigue siendo rápida y exacta.

En la figura 3 se muestra un diagrama de flujo que muestra la estructura de la toma de tiempo. El algoritmo consiste en verificar que se está dentro del rango de iteraciones sin alcanzarlo. De ser así luego se verifica si la variable "caso" es igual a la variable vacío, si esto es así no ocurre nada y se reinicia el ciclo agregando una iteración al contador, si da el caso de que la variable es distinta, se pasa a verificar si esta es igual al contenido de la variable ESPACIO, si lo es, se imprime " ", de no serlo pasa a verificar si es igual a ASTERISCO, de serlo se imprime "***", caso contrario se evalúa si es igual a NEWLINE, si lo es, se agrega un `\n`, y de no serlo se termina el ciclo y se le agrega uno al contador. Cabe mencionar que al imprimirse cualquier dato dado que la variable "caso" es igual a alguno de los if, esto terminará el ciclo y le sumará uno al contador utilizado.

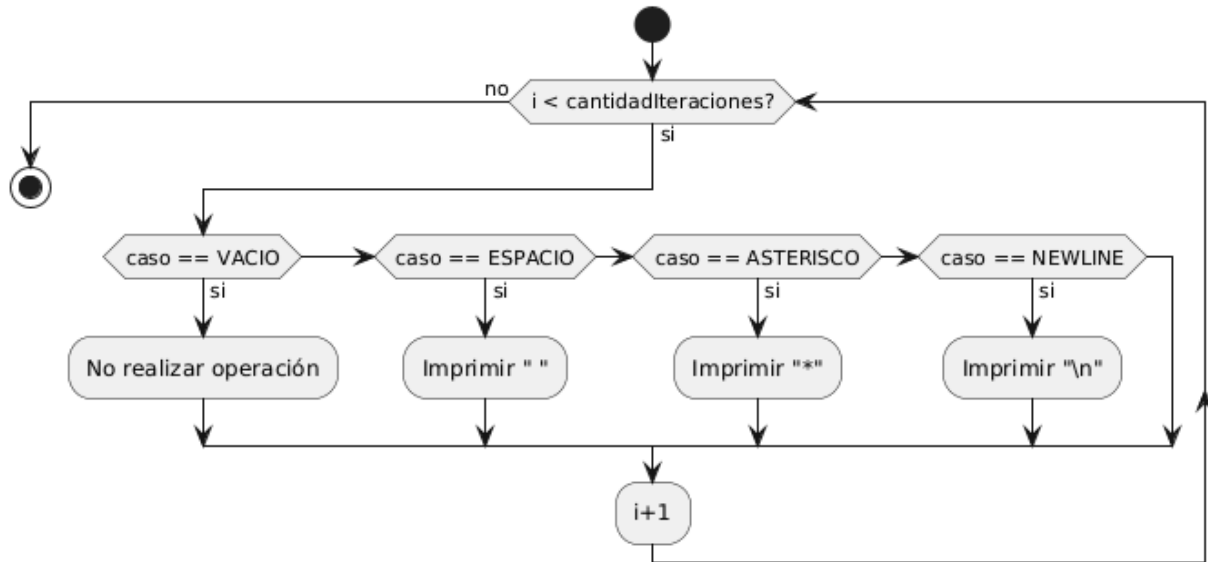


Figura 3

2.3. Hito 3

Para la ejecución del programa en una primera instancia se utilizó un Macbook Neo (chip Apple Silicon A18 pro, 8[GB] de memoria unificada), como compilador se usó Apple clang version 21.0.0

Se realizó la tabla propuesta en las instrucciones de forma vertical con el fin de optimizar el espacio y la legibilidad.

Tabla 1: Tiempos obtenidos tras iteraciones en Apple Clang.

	VACIO	ESPACIO	ASTERISCO	NEWLINE
Tprom 1	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]
Tprom 2	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]
Tprom 3	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]
Tprom 4	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]
Tprom 5	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]

Tprom 6	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]
Tprom 7	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]
Tprom 8	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]
Tprom 9	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]
Tprom 10	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]
Promedio	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]	0.000000 [ns]
Desv. estándar	0	0	0	0

De los resultados obtenidos, asumimos no satisfecho el objetivo del problema dispuesto, con la ayuda de un asistente inteligente (IA), se confirmó la validez del programa dispuesto y se les atribuyen los resultados al compilado y al procesador ocupado para ejecutar el problema, ya que de la función “cicloejemplo()” obtuvimos resultados similares.

De un análisis más profundo podemos afirmar que Apple clang optimiza de forma “muy agresiva” el código dispuesto, no logramos cargar el sistema lo suficiente para obtener tiempos medibles y visibles en las tablas, sin importar la cantidad de iteraciones ni las instrucciones de interpretación directa al programa (gcc -O0 mediciones.cc -o programa).

A modo de evidencia y para asegurar un análisis más significativo se adjunta la siguiente tabla con el registro de las mediciones realizadas por el programa:

Tabla 2: Tiempo de ejecución y resultados obtenidos.

Nº Medición	Mediciones realizadas
1	Tiempo promedio 44.419000 [ns] === Mediciones realizadas ===

	VACIO: Tiempo promedio 0.000000 [ns] ESPACIO: Tiempo promedio 0.000000 [ns] ASTERISCO: Tiempo promedio 0.000000 [ns] NEWLINE: Tiempo promedio 0.000000 [ns]
2	Tiempo promedio 47.918000 [ns] === Mediciones realizadas === VACIO: Tiempo promedio 0.000000 [ns] ESPACIO: Tiempo promedio 0.000000 [ns] ASTERISCO: Tiempo promedio 0.000000 [ns] NEWLINE: Tiempo promedio 0.000000 [ns]
3	Tiempo promedio 48.604000 [ns] === Mediciones realizadas === VACIO: Tiempo promedio 0.000000 [ns] ESPACIO: Tiempo promedio 0.000000 [ns] ASTERISCO: Tiempo promedio 0.000000 [ns] NEWLINE: Tiempo promedio 0.000000 [ns]
4	Tiempo promedio 47.425000 [ns] === Mediciones realizadas === VACIO: Tiempo promedio 0.000000 [ns] ESPACIO: Tiempo promedio 0.000000 [ns] ASTERISCO: Tiempo promedio 0.000000 [ns] NEWLINE: Tiempo promedio 0.000000 [ns]
5	Tiempo promedio 49.242000 [ns] === Mediciones realizadas === VACIO: Tiempo promedio 0.000000 [ns]

	ESPACIO: Tiempo promedio 0.000000 [ns] ASTERISCO: Tiempo promedio 0.000000 [ns] NEWLINE: Tiempo promedio 0.000000 [ns]
6	Tiempo promedio 49.510000 [ns] === Mediciones realizadas === VACIO: Tiempo promedio 0.000000 [ns] ESPACIO: Tiempo promedio 0.000000 [ns] ASTERISCO: Tiempo promedio 0.000000 [ns] NEWLINE: Tiempo promedio 0.000000 [ns]
7	Tiempo promedio 49.081000 [ns] === Mediciones realizadas === VACIO: Tiempo promedio 0.000000 [ns] ESPACIO: Tiempo promedio 0.000000 [ns] ASTERISCO: Tiempo promedio 0.000000 [ns] NEWLINE: Tiempo promedio 0.000000 [ns]
8	Tiempo promedio 50.001000 [ns] === Mediciones realizadas === VACIO: Tiempo promedio 0.000000 [ns] ESPACIO: Tiempo promedio 0.000000 [ns] ASTERISCO: Tiempo promedio 0.000000 [ns] NEWLINE: Tiempo promedio 0.000000 [ns]
9	Tiempo promedio 50.243000 [ns] === Mediciones realizadas === VACIO: Tiempo promedio 0.000000 [ns] ESPACIO: Tiempo promedio 0.000000 [ns] ASTERISCO: Tiempo promedio 0.000000 [ns]

	NEWLINE : Tiempo promedio 0.000000 [ns]
10	Tiempo promedio 48.757000 [ns] === Mediciones realizadas === VACIO: Tiempo promedio 0.000000 [ns] ESPACIO: Tiempo promedio 0.000000 [ns] ASTERISCO: Tiempo promedio 0.000000 [ns] NEWLINE: Tiempo promedio 0.000000 [ns]

Luego de los inconcluyentes datos obtenidos se realizó por segunda vez la medición de los tiempos de ejecución, esta vez en un computador con sistema operativo windows, un procesador Intel(R) Core(TM) i5-10400 2.90 [GHz], 16[GB] de memoria RAM y como compilador se utilizó GCC 15.2.0.

La tabla 3 muestra los resultados obtenidos al ejecutar 10 veces el programa, con la diferencia de que en esta ocasión el código está dispuesto con 1×10^6 iteraciones, ya que, al intentar mantener las iteraciones realizadas en Apple Silicon (se probó con potencia a la 8, 10 e incluso 20), el sistema tardaba demasiado en dar respuesta.

Tabla 3: Tiempos obtenidos en GCC.

	VACIO	ESPACIO	ASTERISCO	NEWLINE
Tprom1	2.000000 [ns]	12938.000000 [ns]	13473.000000 [ns]	12635.000000 [ns]
Tprom2	1.000000 [ns]	15727.000000 [ns]	12877.000000 [ns]	12666.000000 [ns]
Tprom3	1.000000 [ns]	13995.000000 [ns]	13841.000000 [ns]	12491.000000 [ns]
Tprom4	2.000000 [ns]	13999.000000 [ns]	13814.000000 [ns]	12564.000000 [ns]

Tprom5	1.000000 [ns]	14001.000000 [ns]	13727.000000 [ns]	12400.000000 [ns]
Tprom6	1.000000 [ns]	14107.000000 [ns]	13598.000000 [ns]	12913.000000 [ns]
Tprom7	2.000000 [ns]	13935.000000 [ns]	13660.000000 [ns]	12894.000000 [ns]
Tprom8	2.000000 [ns]	14254.000000 [ns]	14296.000000 [ns]	12823.000000 [ns]
Tprom9	1.000000 [ns]	13802.000000 [ns]	13763.000000 [ns]	12348.000000 [ns]
Tprom10	1.000000 [ns]	13723.000000 [ns]	13774.000000 [ns]	12548.000000 [ns]
Promedio	1.4[ns]	14048.100000 [ns]	13628.300000 [ns]	12628.200000 [ns]
Desv. estándar	0.516398	689.878	355.277059	197.525695

A continuación, explicamos por qué los resultados son tan distintos entre sí:

El caso VACÍO es casi instantáneo: Con un promedio de sólo 1.4 ns, este es el tiempo más bajo por lejos. Esto ocurre porque el procesador no tiene que "escribir" nada hacia afuera; solo está dando vueltas en el ciclo, lo cual es una tarea interna muy sencilla para un MacBook moderno.

El gran salto al imprimir (I/O): Notamos que apenas le pedimos al computador imprimir algo (un espacio, un asterisco o un salto de línea), el tiempo sube de 1.4 ns a más de 12,000 ns. Esta diferencia gigante nos demuestra que el procesador es rapidísimo trabajando solo, pero se "demora" mucho más cuando tiene que comunicarse con la pantalla para mostrar un carácter.

Diferencias entre caracteres: Curiosamente, el ESPACIO (14,048 ns) resultó ser un poco más lento en promedio que el ASTERISCO (13,628 ns) y el NEWLINE (12,628 ns). Esto nos dice que, para el sistema operativo, mostrar un espacio vacío requiere el mismo (o incluso un poco más) de esfuerzo que dibujar un símbolo visible.

La estabilidad de los datos (Desviación Estándar):

En el caso VACIO, la desviación es mínima (0.516), lo que significa que el procesador siempre tarda casi lo mismo.

En el caso del ESPACIO, la desviación es la más alta (689.878). Esto indica que el tiempo de impresión fue un poco más "inestable" o irregular, probablemente porque el sistema operativo estaba ocupado con otras tareas pequeñas mientras intentaba imprimir.

3. Conclusión de la evidencia.

Los datos demuestran que el verdadero "cuello de botella" o la parte más lenta de nuestro código no es el ciclo `for` en sí, sino la función `printf`. Como ingenieros en formación, esto nos enseña que las operaciones de Entrada y Salida (I/O) son miles de veces más costosas en tiempo que los cálculos matemáticos internos del procesador.